ProgettoGasCatanzaro

Introduzione

Esaminiamo i dati relativi, alla rete di **gasdotti europea**, i dati sono estratti dal sito del **Agenzia internazionale per l'energia**.

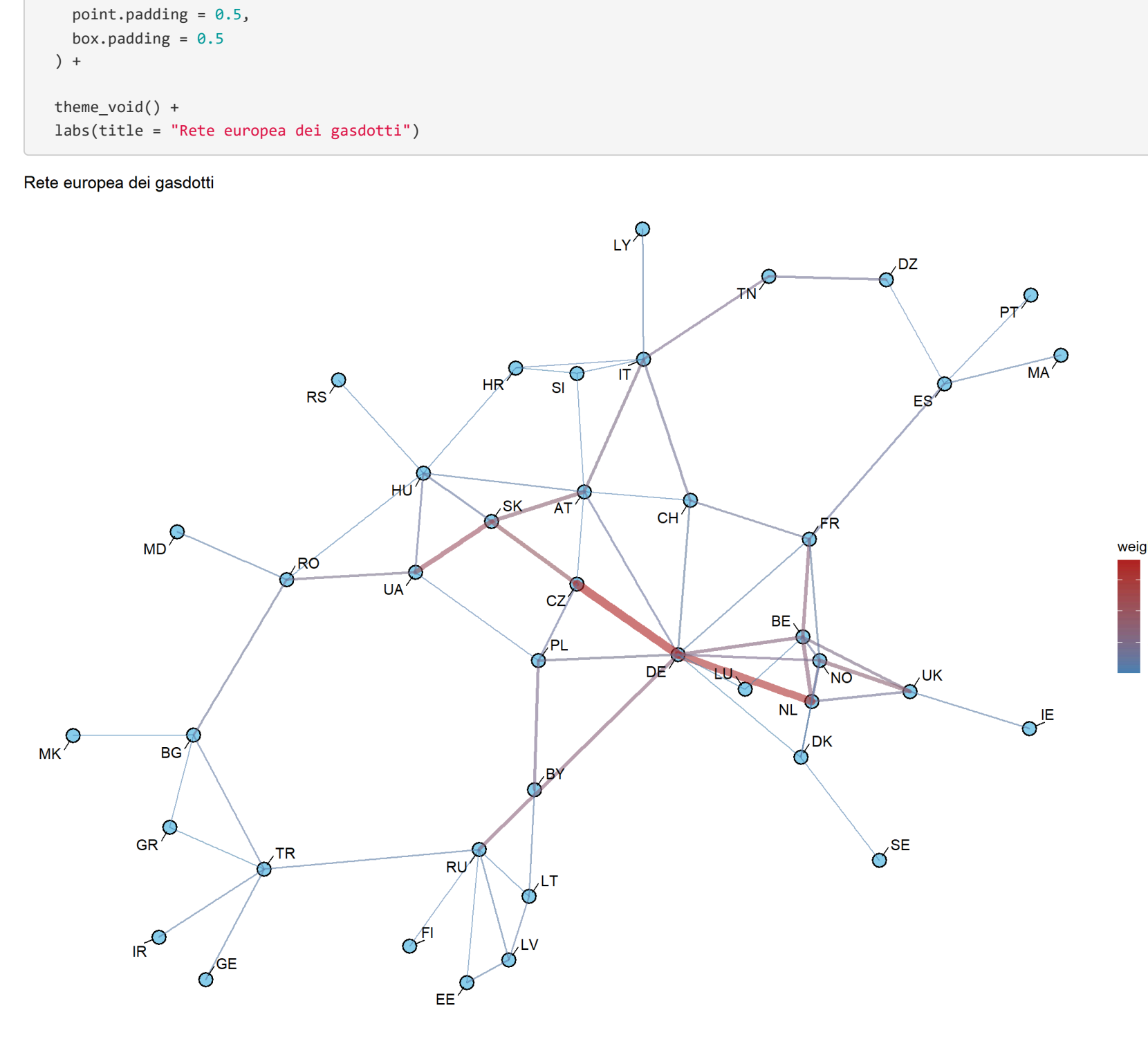
Obbiettivo

Stabilire quali paesi hanno una **centralità maggiore**

Il grafo mostra quali coppie di paesi sono **collegate** da condutture, e la **quantità di gas trasportata**. Il grafo è **indiretto**, significa che la quantità di gas trasportata è cumulativa e reciproca.

```
# setto un layout con i nodi ben distribuiti

set.seed(42)

ig <- as.igraph(g)
lay <- igraph::layout_with_fr(
  ig,
  weights = NA,      # ignora i pesi nel layout
  niter = 8000,      # distanzio i nodi
  grid  = "nogrid"
)


# Disegno il grafo

ggraph(g, layout = lay) +
  geom_node_point(size = 4.5, shape = 21, fill = "skyblue", colour = "black", stroke = 0.8) +
  geom_edge_link(aes(width = weight, colour = weight), alpha = 0.6, lineend = "round") +

   # uso spessore e colore per identificare il peso

  scale_edge_width(range = c(0.4, 3), guide = "none") +
  scale_edge_colour_gradient(low = "steelblue", high = "firebrick") +


  # etichette

  geom_node_text(
    aes(label = short_name),
    repel = TRUE,
    size = 4,
    force = 3,
    point.padding = 0.5,
    box.padding = 0.5
  ) +

  theme_void() +
  labs(title = "Rete europea dei gasdotti")
```

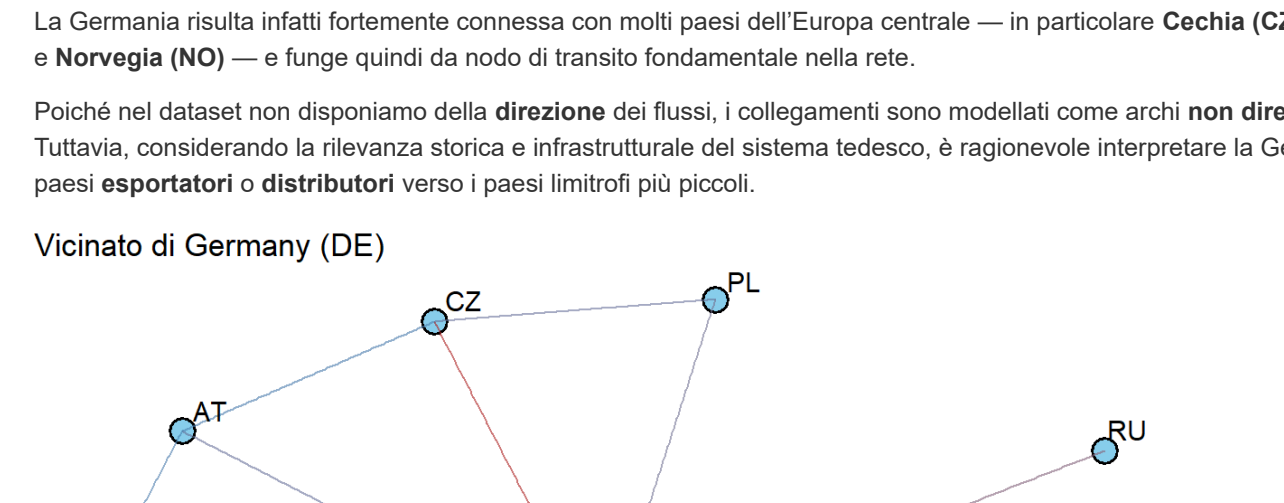
Rete europea dei gasdotti



Osservando il grafo ottenuto, possiamo notare che il principale hub di distribuzione del gas è la **Germania (DE)**.
La Germania risulta infatti fortemente connessa con molti paesi dell'Europa centrale — in particolare **Cechia (CZ)**, **Paesi Bassi (NL)**, **Belgio (BE)** e **Norvegia (NO)** — e funge quindi da nodo di transito fondamentale nella rete.

Poiché nel dataset non disponiamo della **direzione** dei flussi, i collegamenti sono modellati come archi **non diretti**.
Tuttavia, considerando la rilevanza storica e infrastrutturale del sistema tedesco, è ragionevole interpretare la Germania come uno dei principali paesi **esportatori** o **distributori** verso i paesi limitrofi più piccoli.

Vicinato di Germany (DE)



1. Calcolare la centralità Katz e il power della rete

Centralità di Katz

La centralità di Katz misura l'importanza di un nodo in una rete tenendo conto della centralità pre-esistente dei suoi vicini.
In altre parole, **un nodo è tanto più centrale quanto più è collegato a nodi che a loro volta sono centrali.**

Il valore di centralità di un nodo si ottiene **sommando le centralità dei nodi nel suo vicinato**.
Tuttavia, se la rete non è completamente connessa, alcuni nodi potrebbero avere **centralità nulla**.
Per evitare questo problema si introduce un termine **β**, che assegna una centralità minima base a ogni nodo, generalmente è 1 per tutti i nodi, ma a seconda del nostro modello, può derivare da fattori esterni.

Inoltre, per evitare che la somma esploda in valori troppo elevati, si introduce un fattore di attenuazione λ.

La formula della centralità di Katz, per il nodo i:

$$x_i = \lambda \sum_k a_{ki}\, x_k + \beta_k$$

dove:

- a_{ki}: elemento della matrice di adiacenza (= 1 se arco da k → i, = 0 altrimenti)
- x_i: centralità del nodo i
- λ: valore di normalizzazione
- β_k: centralità di base assegnata al nodo k-esimo

Il problema si traduce quindi in un sistema lineare:

$$\mathbf{x} = \lambda \mathbf{x} A + \beta$$

e la soluzione $\mathbf{x}$ fornisce il vettore delle centralità di Katz.

Il sistema **ha soluzione**, quindi l'algoritmo converge a un x **se** :

si sceglie un valore $\lambda < 1/\rho(A)$, cioè lo spettro di A, l'autovalore più grande che possiede A

In quanto:

$$x = \lambda x A + \beta$$

sposto e raccolgo la x:

$$x(I - \lambda A) = \beta.$$

Il sistema ha soluzione **se e solo se** la matrice $I - \lambda A$ è invertibile, quindi se:

$$\det(I - \lambda A) \neq 0.$$

Ora osserviamo che il determinante può essere riscritto come:

$$\det(I - \lambda A) = 0 \quad \Longleftrightarrow \quad \det(A - \tfrac{1}{\lambda} I) = 0.$$

Quindi il determinante è nullo **quando**:

$$\tfrac{1}{\lambda}$$

è un autovalore di A.
*(noi **non** vogliamo sia nullo, altrimenti il sistema non ha soluzione, => **scelto** λ, **allora** 1/λ **non deve essere un autovalore**)* .

Se noi partiamo da 0 a crescere, cerchiamo il primo 1/λ che annulli il determinante:

- **il primo valore che incontriamo 1/λ** che annulla il determinante sarà quindi il più piccolo autovalore per $\det(A - \tfrac{1}{\lambda} I) = 0$
- se 1/λ annulla $\det(A - \tfrac{1}{\lambda} I) = 0$ allora λ annulla $\det(I - \lambda A) = 0$ *(sono la stessa equazione)*
- Allora **λ è l'autovalore più grande di A** (perchè è il primo/più piccolo che l'inversa trova 1/λ, λ sarà il più grande)

Per evitare di scegliere come parametro λ un valore problematico, che mi annulli il determinante e non faccia convergere l'algoritmo

La soluzione migliore è scegliere un valore in :

$$0 < \lambda < \frac{1}{\rho(A)},$$

dove $\rho(A)$ è spettro di A, cioè l'autovalore di modulo massimo di A.

In questo range non troverò autovalori di A, i λ scelti, saranno tutti > del autovalore massimo di A, non c'è ne sono altri dopo
E il **sistema avrà sempre soluzione**, non avrò λ che mi annulleranno il determinante, potrò sempre invertire e trovare una soluzione

Calcolo gli autovalori della matrice di adiacenza A tramite la decomposizione agli autovalori, cioè il metodo classico basato sulla risoluzione dell'equazione caratteristica $\det(A - \lambda I) = 0$, ottenuta numericamente con la funzione `eigen()`. Questo mi permette di individuare $\rho(A)$, l'autovalore di modulo massimo.

Scelto λ, riscrivo la formula di Katz come sistema lineare $(I - \lambda A)x = \beta$ e lo risolvo invertendo la matrice $I - \lambda A$. L'inversione è realizzata con metodi numerici standard (fattorizzazione), tenendo conto che esisterà una soluzione.

```
# Calcolo della centralità di Katz

# Uso le funzioni base di igraph per estrarre la matrice di adiacenza pesata del grafo
ig <- as.igraph(g)
A <- as_adjacency_matrix(ig, attr = "weight")

# Calcolo raggio spettrale
eig <- eigen(A)$values  #calola gli autovalori della matrice
r <- max(abs(eig))      # ne estraggo il massimo

# Scelgo un valore di lambda < 1/r
lambda <- 0.9 / r

# Calcolo la centralità di Katz (ritorna un vettore lungo quanti i nodi)
x <- alpha_centrality(
  ig,
  alpha = lambda,
  exo = 1,
  weights = E(ig)$weight
)

# Mostro la tabella
x_round <- round(x, 3)

tab <- g %>%
  activate(nodes) %>%
  as_tibble() %>%
  mutate(katz = x_round) %>%
  select(name, short_name, katz) %>%
  arrange(desc(katz))

tab
```

```
## # A tibble: 41 × 3
##    name            short_name  katz
##    <chr>           <chr>      <dbl>
##  1 Germany         DE         17.9
##  2 Netherlands     NL         12.9
##  3 Czech Republic  CZ         12.2
##  4 Belgium         BE          9.03
##  5 Norway          NO          7.29
##  6 Slovak Republic SK          6.20
##  7 United Kingdom  UK          5.41
##  8 Russia          RU          4.52
##  9 Austria         AT          4.38
## 10 France          FR          4.35
## # ℹ 31 more rows
```

Conclusione

Come notato visivamente, la germania aveva un ruolo molto forte e dominante, confermato anche dalla centralità di Katz

Power

Il potere all'intero di una rete, è una metrica che misura l'importanza di un nodo in base a quanto deboli solo i suoi vicini.
In altre parole, **un nodo è potente, meno potenti sono i nodi attorno a lui.**
A differenza delle centralità tradizionali, la power si usa per **reti a scambio negativo**, in cui l'importanza aumenta al diminuire di quella degli altri.

Il valore di power di un nodo si ottiene **sommando l'inverso delle power dei nodi nel suo vicinato**.

$$x_i = \sum_k a_{ki} \frac{1}{x_k}$$

dove:

- a_{ki}: elemento della matrice di adiacenza (= 1 se arco da k → i, = 0 altrimenti)
- x_i: la power del nodo i
- x_k: la power del nodo k

Si nota quindi che il problema **non** è in questo caso un semplice sistema lineare, ma:

$$x = x^{\div} A$$

dove il vettore delle inverse delle power di ogni nodo:

$$x^{\div} = \left(\frac{1}{x_1}, \frac{1}{x_2}, \ldots, \frac{1}{x_n}\right)$$

Quindi il sistema calcola x, tramite un **algoritmo iterativo** :

1. **Scelta di un vettore iniziale**
   Si parte da un vettore $x^{(0)}$ (ad esempio un vettore di tutti 1).
2. **Aggiornamento iterativo**
   Per ogni iterazione t, si calcola il nuovo vettore (aggiornando le power solo alla fine):

   $$x^{(t+1)} = x^{(t)\div} A$$

   Continua **fino a convergenza**, cioè quando la differenza tra un vettore e il successivo è piccola

   $$\|x^{(t+1)} - x^{(t)}\| < \varepsilon,$$

Il vettore ottenuto sarà il risultato finale, il vettore power x !

Il'algoritmo converge **quando il grafo è regolarizzabile**

Un grafo **Regolare** è un grafo che :

- un grafo non pesato e indiretto, tutti i nodi hanno grado uguale r

Un grafo **Regolarizzabile** è un grafo :

- un grafo non pesato e indiretto, se posso aggiungere dei pesi interi positivi agli archi, in modo che tutti i nodi abbiano grado pesato uguale r

Un grafo regolare è regolarizzabile sempre (basta assegnare peso 1 agli archi, e ottengo grado pesato uguale a r).

Non tutti i grafi sono regolarizzabili, un grafo è regolarizzabile se vale:

$$Bw = re, \qquad w \geq 1$$

dove:

- B è la matrice di incidenza del grafo (dimensione $n \times m$)
- w è un vettore di lunghezza m contenente i pesi positivi degli archi.
- r è il grado di regolarizzazione, ovvero il valore comune a cui devono essere resi uguali i gradi pesati di tutti i nodi.
- e è il vettore colonna di tutti 1.

La **matrice di incidenza** è una matrice che descrive quali nodi sono collegati da ciascun arco.

- le **righe** corrispondono ai n nodi del grafo
- le **colonne** corrispondono ai m archi, numerati $e_1, e_2, \ldots, e_m$

L'elemento:

$$b_{i,j} = \begin{cases} 1 & \text{se il nodo } i \text{ è uno dei due estremi dell'arco } j \\ 0 & \text{altrimenti} \end{cases}$$

Per esempio, la matrice di incidenza per gli archi e_1 = (1,2) ed e_2 = (1,3) è:

```
      e1  e2  e3  e4
1 -->  1   1   1  0
2 -->  1   0   1  0
3 -->  0   1   1  1
```

Quindi $Bw = re$ significa che, per ogni nodo, **la somma dei pesi scelti w degli archi di cui fa parte** deve risultare uguale allo stesso valore r.
Ottenendo così un vettore di tutti r per ogni nodo, esattamente la definizione di regolarizzabile.

$$\begin{cases} w_1 + w_2 = r \\ w_1 + w_3 = r \\ w_3 = r \end{cases}$$

Se il sistema **non ha soluzione**, il grafo **non è regolarizzabile**.

Genero la matrice di incidenza

```
# Funzione che genera la matrice di incidenza


make_incidence <- function(g) {
  ig <- if (inherits(g, "tbl_graph")) as.igraph(g) else g   # Uso funzioni base di igraph

  n <- vcount(ig) # numero nodi
  m <- ecount(ig) # numero archi

  # ritorna la lista degli archi del grafo, numerati, e la coppia di vertici
  Eends <- ends(ig, E(ig), names = FALSE)

  # Genero la matrice di incidenza
  B <- matrix(0L, nrow = n, ncol = m)
  for (i in seq_len(m)) {                # per ogni arco i-esimo della lista e per B[nodo,arco]
    B[Eends[i, 1], i] <- 1                 #  va nel nodo "from" e mette 1 nel arco i
    B[Eends[i, 2], i] <- 1                 #  stessa cosa per l'altro nodo "to" che fa parte comunque del arco
  }


  # Rinomino i nodi
  rownames(B) <- if (!is.null(V(ig)$name)) V(ig)$name else as.character(seq_len(n))
  colnames(B) <- paste0("e", seq_len(m))

  B
}


B <- make_incidence(g)
B[1:6, 1:9]
```

```
##          e1 e2 e3 e4 e5 e6 e7 e8 e9
## Italy     0  0  0  0  1  0  0  0  0
## Algeria   1  0  0  0  0  0  0  0  0
## Spain     1  1  0  1  0  0  0  0  0
## Belgium   0  0  1  0  0  0  0  1  1
## France    0  1  1  0  0  0  0  0  0
## Portugal  0  0  0  1  0  0  0  0  0
```

Ora quindi possiamo mettere questa matrice a sistema $Bw = re$, DEVO **trovare W e r tale che il sistema abbia soluzione**.
Se riesco a trovarli, se esistono **allora il grafo è regolarizzabile**

Codice per verificare se un grado è regolarizzabile

```
# Codice per VERIFICARE SE ESISTONO w e r CHE PORTINO A UNA SOLZIONE DEL SISTEMA
# SE ESISTONO VENGONO RITORNATI w e r
# Altrimenti se torna NULL => il grafo non è regolarizzabile

# librerie per risolvere sistemi lineari complessi
library(lpSolve)
library(lpSolveAPI)


regularify <- function(g) {
  ig <- if (inherits(g, "igraph")) g else igraph::as.igraph(g)  # uso funzioni base di igraph

  n <- vcount(ig) # numero nodi
  m <- ecount(ig) # numero archi

  # Usa la tua funzione già definita
  B <- make_incidence(ig)   # n x m

  # max 0  s.t.  [B | -1] * (w_hat, r) = -deg
  # con vincoli interni w = w_hat + 1  => w >= 1
  obj <- rep(0, m + 1)                          # obiettivo nullo (solo fattibilità)
  con <- cbind(B, rep(-1, n))                   # matrice dei vincoli
  dir <- rep("=", n)                            # uguaglianze
  rhs <- -igraph::degree(ig, mode = "all")      # termine noto

  sol <- lp(direction   = "max",
            objective.in = obj,
            const.mat    = con,
            const.dir    = dir,
            const.rhs    = rhs)

  # status 0 = fattibile/ottimo trovato
  if (sol$status == 0) {
    s <- sol$solution
    w <- s[1:m] + 1          # spostamento: impone w >= 1
    r <- s[m + 1]            # grado pesato comune
    return(list(weights = w, degree = r))
  } else {
    return(NULL)             # non regolarizzabile (LP infeasible)
  }
}

regularify(g)
```

```
## NULL
```

Problema, il nostro grafo è risultato **NON REGOLARIZZABILE** (regularify(g) è risultata NULL), l'algoritmo iterativo non converge mai
E' possibile aggiungere una **perturbazione diagonale** al grafo, cioè modificare la matrice di adiacenza aggiungendo un **self-loop a ogni nodo**, se pesata, come nel nostro caso, un **piccolo peso** ε>0 (es. ε = 0.1)

La nuova matrice di adiacenza quindi sarà :

$$A_\varepsilon^D = A + \varepsilon I$$

Codice per perturbare il grafo, e verifico se il grafo perturbato è regolarizzabile :

```
# matrice di adiacenza originale
A  <- igraph::as_adjacency_matrix(g, sparse = FALSE)

# aggiungo un perso espilon alla diagonale = 0.1
eps <- 0.1
AI  <- A + diag(eps, nrow(A))

# NUOVO GRAFO PERTURBATO
g_eps <- graph_from_adjacency_matrix(AI, mode = "undirected", weighted = TRUE)

# controllo se ora è regolarizzabile
regularify(g_eps)
```

```
## $weights
##   [1]  1  1  1  1  1 13  1  1  1 17  1  1  1 15  1  1  1  1  1 10  4  1  5  1 10
##  [26] 18  1  1  2  1 14  1  1 11  1  2  1  1  1  1 16  1  1 16 14  1  1  1  1  1
##  [51]  1 15  1  1 14  1  1  1  1 13  1  1  1  1  1  8  1  1  1  1  1  1  2  1  1
##  [76]  1 15 18  1  1  1  1  1 14  1  1  1 15  4  1 16  1  6  1  1 12  2  7  1  1
## [101] 15 18  1  3  6  1  4  4  5  4
##
## $degree
## [1] 20
```

Questo accade in quanto il piccolo self-loop aumenta leggermente il grado pesato di ogni nodo.
In questo modo tutti i nodi possono raggiungere lo stesso grado pesato e il sistema $Bw = re$ diventa risolvibile

Ora possiamo calcolare il vettore della power

Adesso che il grafo è regolarizzabile, l'algorirmo iterativo per calcolare la power andrà a **convergenza**,
posso calcolare la power, posso estrarre il **vettore della power x** :

```
 # Funzione per calcolare la power

# g_eps   # Grafo SICURAMENTE REGOLARIZZABILE, possiamo calcolarne la power

# codice appunti
power_g <- function(g, t) {

  ig <- if (inherits(g, "igraph")) g else igraph::as.igraph(g)  # uso le funzioni di igraph base

  A <- as.matrix(igraph::as_adj(ig, attr = "weight", sparse = FALSE))
  A[is.na(A)] <- 0

  n = dim(A)[1]

  # x_2k
  x0 = rep(0, n)
  # x_2k+1
  x1 = rep(1, n)
  # x_2k+2
  x2 = rep(1, n)

  diff = 1
  eps  = 1 / 10^t
  iter = 0

  while (diff > eps) {
    x0 = x1
    x1 = x2
    x2 = (1 / x2) %*% A
    diff = sum(abs(x2 - x0))
    iter = iter + 1
  }

  # alpha x2 = (1/x2) A
  alpha = ((1 / x2) %*% A[, 1]) / x2[1]

  # soluzione finale: sqrt(alpha) * x2
  x2 = sqrt(alpha) %*% x2

  return(list(vector = as.vector(x2), iter = iter))
}



# CALCOLO LE POWER
res_power <- power_g(g_eps, t = 6)
pvec       <- as.numeric(res_power$vector)


# AGGIUNGO LA POWER COME ATTRIBUTO DEI NODI AL GRAFO

nodes_orig <- g %>%
  activate(nodes) %>%
  as_tibble() %>%
  select(name, short_name)


nodes_eps <- tibble(
  name  = V(g_eps)$name,
  power = pvec
)


df_power <- nodes_eps %>%
  left_join(nodes_orig, by = "name") %>%
  select(name, short_name, power) %>%
  arrange(desc(power))


print(df_power, n = 7)
```

```
## # A tibble: 41 × 3
##   name     short_name power
##   <chr>    <chr>      <dbl>
## 1 Turkey   TR          7.41
## 2 Spain    ES          6.39
## 3 Germany  DE          6.31
## 4 Italy    IT          6.01
## 5 Hungary  HU          5.01
## 6 Russia   RU          4.96
## 7 Bulgaria BG          4.61
## # ℹ 34 more rows
```

2/3. Analizziamo i risultati di centralità e power ottenuti

```
## # A tibble: 41 × 4
##    name            short_name  katz power
##    <chr>           <chr>      <dbl> <dbl>
##  1 Italy           IT          2.20 6.01
##  2 Algeria         DZ          1.20 1.12
##  3 Spain           ES          1.46 6.39
##  4 Belgium         BE          9.03 3.80
##  5 France          FR          4.35 1.88
##  6 Portugal        PT          1.04 0.404
##  7 Slovak Republic SK          6.20 1.89
##  8 Austria         AT          4.38 3.28
##  9 Latvia          LV          1.31 2.11
## 10 Lithuania       LT          1.19 1.63
## # ℹ 31 more rows
```

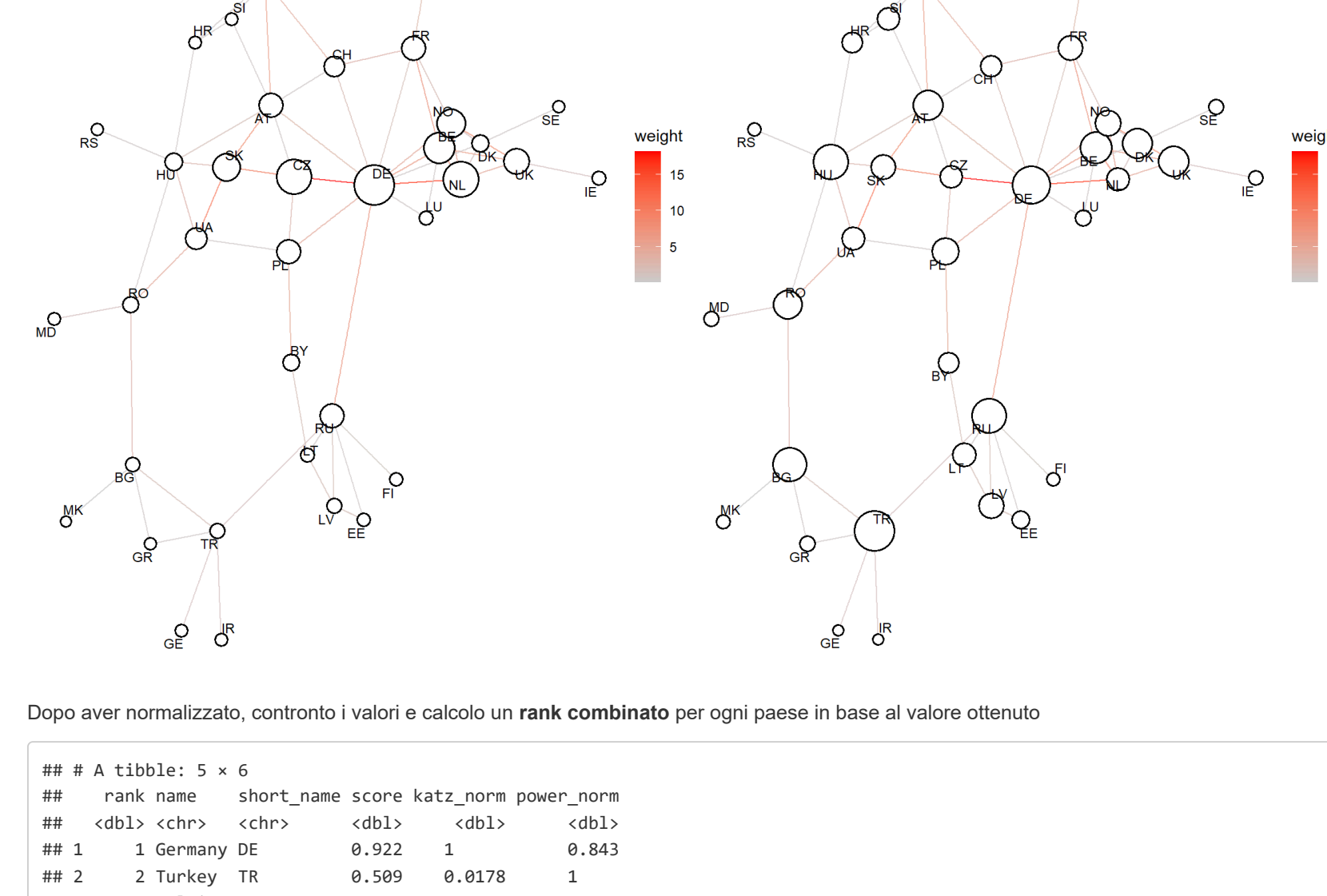
Prima di tutto analizziamo se le 2 misure hanno lo stesso trend
cioè se una misura è alta anche l'altra è alta e viceversa

La correlazione tra le 2 misure è :

```
## Correlazione Spearman tra Katz e Power: 0.7253
```

E' un risultato molto grande, le 2 misure vanno quasi sempre nella stessa direzione

```
library(gridExtra)

ig_full <- as.igraph(g)
set.seed(123)
lay <- igraph::layout_with_fr(ig_full, weights = NA, niter = 8000, grid = "nogrid")

plot_katz <-
  ggraph(g, layout = lay) +
  geom_edge_link(aes(colour = weight), alpha = 0.7, lineend = "round") +
  scale_edge_colour_gradient(low = "grey80", high = "red") +
  geom_node_point(aes(size = katz), shape = 21, fill = "white",
                  colour = "black", stroke = 0.8) +
  geom_node_text(aes(label = short_name), repel = TRUE, size = 3.2) +
  scale_size_continuous(range = c(3, 12), guide = "none") +
  theme_void() +
  labs(title = "Centralità di Katz")

plot_power <-
  ggraph(g, layout = lay) +
  geom_edge_link(aes(colour = weight), alpha = 0.7, lineend = "round") +
  scale_edge_colour_gradient(low = "grey80", high = "red") +
  geom_node_point(aes(size = power), shape = 21, fill = "white",
                  colour = "black", stroke = 0.8) +
  geom_node_text(aes(label = short_name), repel = TRUE, size = 3.2) +
  scale_size_continuous(range = c(3, 12), guide = "none") +
  theme_void() +
  labs(title = "Power")

grid.arrange(plot_katz, plot_power, ncol = 2)
```


Centralità di Katz

Power

Dopo aver normalizzato, confronto i valori e calcolo un **rank combinato** per ogni paese in base al valore ottenuto

```
## # A tibble: 5 × 6
##    rank name    short_name score katz_norm power_norm
##   <dbl> <chr>   <chr>      <dbl>     <dbl>      <dbl>
## 1     1 Germany DE         0.922    1          0.843
## 2     2 Turkey  TR         0.509    0.0178     1
## 3     3 Belgium BE         0.480    0.475      0.486
## 4     4 Spain   ES         0.441    0.0269     0.856
## 5     5 Italy   IT         0.436    0.0710     0.800
```

```
## # A tibble: 5 × 6
##    rank name    short_name score katz_norm power_norm
##   <dbl> <chr>   <chr>      <dbl>     <dbl>      <dbl>
## 1     1 Germany DE         0.922    1          0.843
## 2     2 Turkey  TR         0.509    0.0178     1
## 3     3 Belgium BE         0.480    0.475      0.486
## 4     4 Spain   ES         0.441    0.0269     0.856
## 5     5 Italy   IT         0.436    0.0710     0.800
```

Conclusioni

Dall'analisi combinata delle due misure — la **centralità di Katz** e la **power** — emergono differenze importanti che mostrano come i due indici catturino aspetti diversi della rete dei gasdotti europei.

In entrambe le misure la **Germania** risulta nettamente al primo posto: è il nodo più connesso, più influente e quello da cui dipende la maggior parte dei paesi vicini. La sua posizione dominante è confermata sia dalla centralità strutturale (Katz), sia dalla capacità di "dominare" nodi più deboli (power).

Tuttavia, non tutti i paesi mostrano un comportamento coerente tra le due metriche.
Ad esempio:

- **Paesi Bassi (NL)**: molto alti in Katz, ma più bassi in power. Significa che sono collegati a nodi centrali, ma non esercitano lo stesso livello di "forza" nei confronti dei loro vicini.
- **Slovacchia (SK)** e **Cechia (CZ)**: ottimi valori di power ma valori di Katz relativamente più bassi. Sono nodi che dipendono da pochi vicini forti, ma che a loro volta si trovano in posizioni in cui "domina" molti nodi deboli.

All'estremo opposto, i paesi come **Macedonia**, **Georgia**, **Iran**, **Serbia**, mostrano valori molto bassi in entrambe le misure: sono periferici, scarsamente collegati e con poca influenza sulla rete.

In sintesi, **Katz evidenzia la centralità strutturale**, mentre **power evidenzia la forza contrattuale** in una rete a scambio negativo.
La combinazione delle due misure mostra chiaramente che la Germania occupa una posizione dominante sotto entrambi i punti di vista, mentre altri paesi come Paesi Bassi o Slovacchia assumono ruoli differenti a seconda del tipo di centralità considerata.